

H.T.NO:2303A51653

Batch 28

Task 1:

Prompt:

Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.

```
Assignment8.1.py > ...
1 def is_strong_password(password):
2     """
3     Validates if a password is strong.
4     Requirements:
5     - At least 8 characters long
6     - Contains at least one uppercase letter
7     - Contains at least one lowercase letter
8     - Contains at least one digit
9     - Contains at least one special character (!@#$$%^&*)
10    """
11    if len(password) < 8:
12        return False
13
14    has_upper = any(c.isupper() for c in password)
15    has_lower = any(c.islower() for c in password)
16    has_digit = any(c.isdigit() for c in password)
17    has_special = any(c in "!@#$$%^&*" for c in password)
18
19    return has_upper and has_lower and has_digit and has_special
20
21
22 # Test Cases
23 assert is_strong_password("Pass123!") == True, "Should accept valid strong password"
24 assert is_strong_password("weak") == False, "Should reject password shorter than 8 characters"
25 assert is_strong_password("NoDigit!") == False, "Should reject password without digits"
26 assert is_strong_password("NoSpecial123") == False, "Should reject password without special character"
27 assert is_strong_password("ALLUPPER123!") == False, "Should reject password without lowercase letters"
28 assert is_strong_password("alllower123!") == False, "Should reject password without uppercase letters"
29
```

```
print("All test cases passed!")
```

```
Abhishek_Cherry\OneDrive\Documents\Desktop\AI Assistant Coding Assignments'; & 'c:\Users\Abhishek_Cherry\OneDrive\Documents\Desktop\AI Assistant Coding Assignments\Assignment8.1.py'
Abhishek_Cherry\OneDrive\Documents\Desktop\AI Assistant Coding Assignments\Assignment8.1.py
All test cases passed!
PS C:\Users\Abhishek_Cherry\OneDrive\Documents\Desktop\AI Assistant Coding Assignments>
```

Task 2:

Prompt:

Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

```
def classify_number(n):
    """
    Classifies a number as Positive, Negative, or Zero, and handles invalid inputs.
    Implements classification logic using a loop over conditions.

    Args:
        n: The number to classify. Can be int, float, string, or None.

    Returns:
        A string: "Positive", "Negative", "Zero", or "Invalid Input".
    """
    if n is None or not isinstance(n, (int, float)):
        return "Invalid Input"

    # using a loop to iterate through classification conditions
    classification_rules = [
        (lambda x: x > 0, "Positive"),
        (lambda x: x < 0, "Negative"),
        (lambda x: x == 0, "Zero")
    ]

    for condition_func, result_str in classification_rules:
        if condition_func(n):
            return result_str

    # This line should ideally not be reached if rules are exhaustive for valid numbers
    return "Invalid Input" # Fallback for unexpected cases or non-numeric types

# AI-generated assert test cases
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
assert classify_number(0) == "Positive" # Boundary condition
assert classify_number(-1) == "Negative" # Boundary condition
assert classify_number("text") == "Invalid Input" # Invalid input: string
assert classify_number(None) == "Invalid Input" # Invalid input: None

assert classify_number(None) == "Invalid Input" # Invalid input: None
assert classify_number(3.14) == "Positive"
assert classify_number(-2.7) == "Negative"
print("All test cases passed.")

... All test cases passed.
```

Task 3:

Prompt:

Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

```

import re

def is_anagram(str1, str2):
    """
    Checks if two strings are anagrams of each other.
    Ignores case, spaces, and punctuation.

    Args:
        str1 (str): The first string.
        str2 (str): The second string.

    Returns:
        bool: True if the strings are anagrams, False otherwise.
    """
    # Clean strings: convert to lowercase and remove non-alphanumeric characters
    clean_str1 = re.sub(r'[^a-z0-9]', '', str1.lower())
    clean_str2 = re.sub(r'[^a-z0-9]', '', str2.lower())

    # Anagrams must have the same length after cleaning
    if len(clean_str1) != len(clean_str2):
        return False

    # Sort characters and compare
    return sorted(clean_str1) == sorted(clean_str2)

# AI-generated assert test cases
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True # Example from prompt
assert is_anagram("Madam Curie", "Meitnerium") == False # Different letters, AI-generated
assert is_anagram("A decimal point", "I'm a dot in place") == True # With spaces and punctuation, AI-generated
assert is_anagram("Debit card", "Bad credit") == True # AI-generated
assert is_anagram("", "") == True # Empty strings, AI-generated edge case
assert is_anagram("a", "A") == True # Case-insensitivity, AI-generated edge case
assert is_anagram("apple", "apply") == False # AI-generated
print("All anagram test cases passed!")

```

All anagram test cases passed!

Task 4:

Prompt:

Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

```
class Inventory:
    """
    A simple inventory management system.
    """
    def __init__(self):
        self.stock = {}

    def add_item(self, name, quantity):
        """
        Adds a specified quantity of an item to the inventory.
        If the item does not exist, it's added with the given quantity.
        Quantity must be non-negative.
        """
        if not isinstance(name, str) or not name.strip():
            raise ValueError("Item name must be a non-empty string.")
        if not isinstance(quantity, (int, float)) or quantity < 0:
            raise ValueError("Quantity must be a non-negative number.")

        self.stock[name] = self.stock.get(name, 0) + quantity

    def remove_item(self, name, quantity):
        """
        Removes a specified quantity of an item from the inventory.
        If the item does not exist or quantity to remove exceeds current stock,
        the item's stock will be set to 0.
        Quantity must be non-negative.
        """
        if not isinstance(name, str) or not name.strip():
            raise ValueError("Item name must be a non-empty string.")
        if not isinstance(quantity, (int, float)) or quantity < 0:
            raise ValueError("Quantity must be a non-negative number.")

        if name in self.stock:
            self.stock[name] = max(0, self.stock[name] - quantity)
        # If item not in stock, no action needed, effectively stock remains 0.
```

```

# If item not in stock, no action needed, effectively stock remains 0.

def get_stock(self, name):
    """
    Returns the current stock level of an item.
    Returns 0 if the item is not in the inventory.
    """
    if not isinstance(name, str) or not name.strip():
        raise ValueError("Item name must be a non-empty string.")
    return self.stock.get(name, 0)

# AI-generated assert test cases
inv = Inventory()

# 1. Add item and check stock
inv.add_item("Laptop", 5)
assert inv.get_stock("Laptop") == 5, f"Expected 5 Laptops, got {inv.get_stock('Laptop')}}"

# 2. Add more of the same item
inv.add_item("Laptop", 3)
assert inv.get_stock("Laptop") == 8, f"Expected 8 Laptops, got {inv.get_stock('Laptop')}}"

# 3. Remove some items
inv.remove_item("Laptop", 2)
assert inv.get_stock("Laptop") == 6, f"Expected 6 Laptops, got {inv.get_stock('Laptop')}}"

# 4. Remove more than available (should result in 0)
inv.remove_item("Laptop", 10)
assert inv.get_stock("Laptop") == 0, f"Expected 0 Laptops, got {inv.get_stock('Laptop')}}"

# 5. Add a new item
inv.add_item("Keyboard", 10)
assert inv.get_stock("Keyboard") == 10, f"Expected 10 Keyboards, got {inv.get_stock('Keyboard')}}"

# 6. Check stock of a non-existent item
assert inv.get_stock("Mouse") == 0, f"Expected 0 Mouse, got {inv.get_stock('Mouse')}}"

# 7. Add zero quantity (should not change existing stock or add new if not present)
inv.add_item("Keyboard", 0)
assert inv.get_stock("Keyboard") == 10, f"Expected 10 Keyboards after adding 0, got {inv.get_stock('Keyboard')}}"
inv.add_item("Monitor", 0)
assert inv.get_stock("Monitor") == 0, f"Expected 0 Monitor after adding 0, got {inv.get_stock('Monitor')}}"

# 8. Remove zero quantity (should not change stock)
inv.add_item("Desk", 5)
inv.remove_item("Desk", 0)
assert inv.get_stock("Desk") == 5, f"Expected 5 Desks after removing 0, got {inv.get_stock('Desk')}}"

# 9. Remove non-existent item (should remain 0)
inv.remove_item("Headphones", 3)
assert inv.get_stock("Headphones") == 0, f"Expected 0 Headphones after removing non-existent, got {inv.get_stock('Headphones')}}"

# 10. Test with float quantities
inv.add_item("Pencil", 10.5)
assert inv.get_stock("Pencil") == 10.5, f"Expected 10.5 Pencils, got {inv.get_stock('Pencil')}}"
inv.remove_item("Pencil", 2.5)
assert inv.get_stock("Pencil") == 8.0, f"Expected 8.0 Pencils, got {inv.get_stock('Pencil')}}"

# 11. Test with invalid inputs for add_item
try:
    inv.add_item("", 10)
    assert False, "Expected ValueError for empty item name"
except ValueError as e:
    assert str(e) == "Item name must be a non-empty string."

```

I
ls



```
try:
    inv.add_item("Item", -5)
    assert False, "Expected ValueError for negative quantity"
except ValueError as e:
    assert str(e) == "Quantity must be a non-negative number."

# 12. Test with invalid inputs for remove_item
try:
    inv.remove_item("", 10)
    assert False, "Expected ValueError for empty item name"
except ValueError as e:
    assert str(e) == "Item name must be a non-empty string."

try:
    inv.remove_item("Item", -5)
    assert False, "Expected ValueError for negative quantity"
except ValueError as e:
    assert str(e) == "Quantity must be a non-negative number."

# 13. Test with invalid inputs for get_stock
try:
    inv.get_stock("")
    assert False, "Expected ValueError for empty item name"
except ValueError as e:
    assert str(e) == "Item name must be a non-empty string."

print("All Inventory class test cases passed!")
```

... All Inventory class test cases passed!

Task 5:

Prompt:

Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

```

from datetime import datetime

def validate_and_format_date(date_str):
    """
    Validates a date string in 'MM/DD/YYYY' format and converts it to 'YYYY-MM-DD'.
    Handles invalid dates.

    Args:
        date_str (str): The date string to validate and format.

    Returns:
        str: The formatted date 'YYYY-MM-DD' or 'Invalid Date' if validation fails.
    """
    try:
        # Attempt to parse the date string with the expected format
        # This will raise a ValueError for invalid dates (e.g., 02/30/2023)
        # or incorrect format (e.g., 'YYYY-MM-DD')
        parsed_date = datetime.strptime(date_str, "%m/%d/%Y")
        # If parsing is successful, format it to the desired output format
        return parsed_date.strftime("%Y-%m-%d")
    except (ValueError, TypeError):
        # Catch ValueError for invalid date components or incorrect format,
        # and TypeError for non-string inputs.
        return "Invalid Date"

# AI-generated assert test cases
assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date" # Invalid day
assert validate_and_format_date("01/01/2024") == "2024-01-01"
assert validate_and_format_date("12/31/2023") == "2023-12-31" # Valid end-of-year date
assert validate_and_format_date("03/15/2023") == "2023-03-15" # Valid date
assert validate_and_format_date("13/01/2023") == "Invalid Date" # Invalid month
assert validate_and_format_date("10-15-2023") == "Invalid Date" # Incorrect format separator
assert validate_and_format_date("2023/10/15") == "Invalid Date" # Incorrect format order
assert validate_and_format_date(None) == "Invalid Date" # Non-string input
assert validate_and_format_date("not a date") == "Invalid Date" # Non-date string

```

```
print("All date validation and formatting test cases passed!")
```

All date validation and formatting test cases passed!
